

SELF-HOSTED AGENT ORCHESTRATOR

SelfHostedContextCrafter

Scoped planning and code auditing
on local infrastructure

Concept document — Idea proposal

June 25, 2026

A system that separates *thinking* (planning with verifiable context),
doing (building with a human in control), and *verifying* (auditing against the source),
all running on your own hardware.

Executive summary

SelfHostedContextCrafter is a **self-hosted** LLM agent orchestrator designed for teams developing software under real-world constraints — specific hardware, business requirements, and regulations — who cannot (or do not want to) send that information to a cloud AI provider.

The core idea is simple but uncommon among today's tools: **the agent never writes production code**. Its only job is to read and digest all relevant technical documentation (hardware datasheets, requirements, regulations, architecture context) and produce a scoped, traceable *advanced plan*. That plan is what a human — supported by their own code agent — uses to build. Once the code exists, a second mode of the same system **audits** it against the plan and the original documentation, returning a per-module score, suggested changes, and specific warnings.

The entire cycle runs on local hardware, with self-hosted LLM models, which makes this approach viable in contexts where the confidentiality of hardware, code, or regulations is a requirement, not a preference.

1 The problem

Current code assistants are powerful at generating text, but they share three recurring weaknesses in projects with real-world constraints:

- **They lose the original context.** They generate reasonable-looking code in the moment, but without clear traceability back to the requirement, regulation, or hardware datasheet that originated it.
- **They hallucinate when context is scattered.** The more fragmented the documentation (manufacturer PDFs, regulations, internal specs), the easier it is for the model to fill gaps with assumptions.
- **They require sending sensitive information to external services.** Proprietary hardware, internal regulations, or source code cannot always leave the organization.

On top of this there is a **control** problem: when an agent generates and audits its own code in the same step, there is no real independent verification — it is the same reasoning reviewing itself.

2 The core idea

SelfHostedContextCrafter splits the development cycle into three distinct roles, each with a scoped responsibility:

1. Plan (Planner Mode)

Ingests all available technical documentation and turns it into an advanced project plan. It does not write a single line of production code: its deliverable is the project structure and a plan scoped to what the documentation actually says.

2. Build (Human + code agent)

A developer, supported by their code agent of choice, executes the plan. This is where the software is actually written — with human judgment, and with the scoped plan as a guide, not a straitjacket.

3. Audit (Auditer Mode)

The same system, retaining memory of everything ingested during the planning phase, reviews the resulting code against the original plan and the source documentation. It returns a per-module score, suggested changes, and specific warnings.

The key is not any one of the three steps on its own, but that all three share the **same retained knowledge base** (RAG). The auditor does not audit in a vacuum: it audits against exactly the same regulatory and technical context that generated the plan, which makes its warnings traceable back to the original source.

3 Conceptual flow architecture

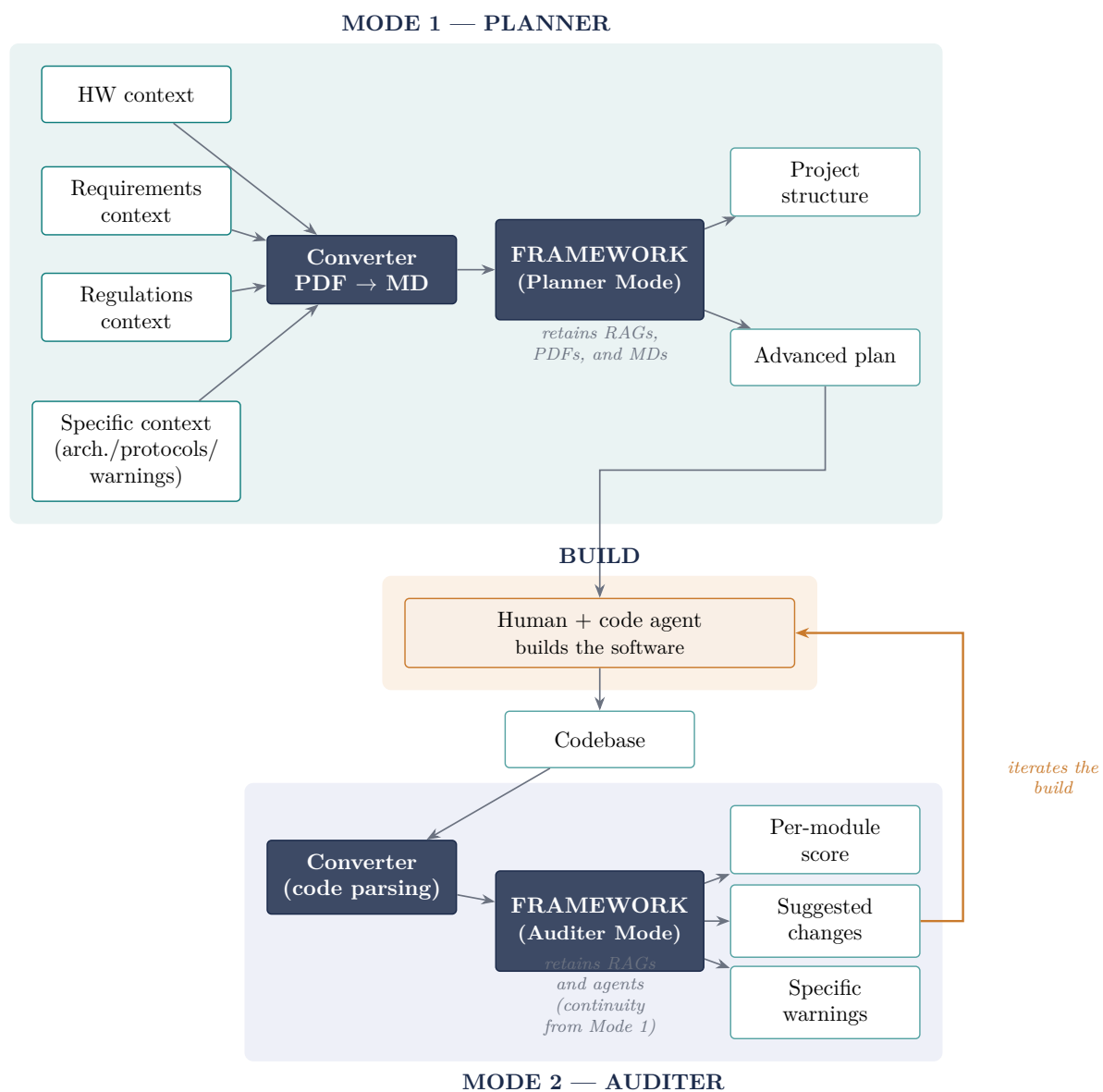


Figure 1: Conceptual flow: context flows from the Planner to the Build stage and from there to the Auditor, without the agent ever touching production code directly. Suggested changes and warnings feed back into the build cycle.

3.1 Reading the diagram

- **Mode 1** normalizes four distinct context sources (hardware, requirements, regulations, specific context) through a single PDF → MD converter, so the Framework always works on homogeneous text. The output is not code: it is a project structure and an advanced plan, and the Framework retains the source material as a persistent knowledge base.
- **The build stage** is deliberately human. The advanced plan and the retained context feed the developer and their code agent of choice, but the implementation decisions remain in human hands.
- **Mode 2** receives the resulting code (via a dedicated converter geared toward code parsing) and the same advanced plan. The Framework in this mode does not start from scratch: it retains the RAGs and agents from Mode 1, so it can audit the code against the original documentation, not just against the plan.

4 Design principles

Self-hosted by default

LLM models running on local hardware. No hardware datasheet, internal regulation, or line of code needs to leave your own infrastructure for the system to work.

The agent plans, the human builds

Separating plan generation from code generation reduces the hallucination surface where it hurts most: architectural decisions. The code is still written by whoever holds the judgment and final responsibility.

Scoped planning, not generic templates

The advanced plan does not come from the model's general knowledge: it comes from what the ingested documentation actually says. That makes it narrower, but also more reliable and easier to justify in a real audit.

Audit with memory, not blind audit

By retaining RAGs and agents across modes, the auditor evaluates the code against the same source of truth that generated the plan — not against a new, potentially different interpretation of the same problem.

5 Where this idea fits

- **Firmware or embedded systems** teams working with manufacturer datasheets and safety regulations (industrial, automotive, medical) who cannot upload those documents to a cloud LLM.
- Organizations with **data sovereignty requirements** that already have, or are willing to invest in, local compute for model inference.

- Projects where the **cost of a regulatory misinterpretation** is high, and where having a plan traceable to the source documentation has value in itself, beyond development speed.

6 In one sentence

“An orchestrator that separates thinking, building, and verifying — one that never assumes what the documentation doesn’t say, and never leaves the hardware it runs on.”